Developing a parallel version of Mandelbrot using OpenMP

Almir Moreira da Silva Neto[1], **Luciano Araújo Dourado Filho**[1]

[1] PGCC - Computer Science Postgraduate Program
Universidade Estadual de Feira de Santana
Bahia, Brazil

almirneto338@gmail.com[1],

lucianoadfilho@gmail.com[1]

1. Introduction

This report presents a comparative analysis of performance between two distinct computational paradigms, serial and parallel, for computation and visualization of the Mandelbrot set through the escape-time algorithm. The serial version was implemented in a non-optimized way for executing in a single thread, whereas the parallel version was developed with the OpenMP API for the C programming language and optimized for executing in multiple threads.

1.1. Escape-Time Algorithm

The Mandelbrot set is built by iterating over the function:

$$z_{n+1} = z_n^2 + c$$

, where $c(x, y) = x + iy$, is a complex number mapped from pixel coordinates (x, y) into the complex domain. If we establish $x_{min}, x_{max}, y_{min}, y_{max}$: the horizontal and vertical axes lengths for the complex plane, a $width$ and $height$ for the output image resolution, we can initialize $z = 0$ and iteratively update it through the formula above for a predetermined number of iterations $maxIter$. The process then consists of checking whether the sequence remains bounded or escapes to infinity i.e., $|z| > 2$, and then coloring each pixel proportionally to the number of iterations taken to escape. A more concise description of the algorithm is described in the Algorithm 1 and will be presented in more detail in the following sections.

Although the algorithm may appear to be sequential, due to the recursive nature of updating z, as it operates at pixel level, it can be optimized to run in a multi-thread parallel environment, by parallelizing pixel-wise computations.

2. Preliminaries

There are different parallelization strategies that can be performed depending on problem's nature and processing environment related constraints such as: number of processors, disk size and memory available. The Master/Worker, for example, is a paradigm in which the master node is assigned responsible for breaking a task and distributing through the workers as they perform the assigned sub-tasks. The master node is previously assigned (and identified), and sometimes can also be responsible for aggregating the results (reducing operations) or managing communication between workers. There are

Algorithm 1 Escape Time Algorithm for Mandelbrot Set

procedure COMPUTEMANDELBROT($x_{\text{min}}, x_{\text{max}}, y_{\text{min}}, y_{\text{max}}$, width, height, maxIter)
  for $i = 0$ to width $- 1$ **do**
    for $j = 0$ to height $- 1$ **do**
      $x \leftarrow x_{\text{min}} + i \times \frac{x_{\text{max}} - x_{\text{min}}}{\text{width}}$
      $y \leftarrow y_{\text{min}} + j \times \frac{y_{\text{max}} - y_{\text{min}}}{\text{height}}$
      $c \leftarrow x + iy$
      $z \leftarrow 0$
      $n \leftarrow 0$
      while $|z| \leq 2$ and $n <$ maxIter **do**
        $z \leftarrow z^2 + c$
        $n \leftarrow n + 1$
      end while
      Store n as the color value for pixel (i, j)
    end for
  end for
end procedure

many other classical distributed computation paradigms, e.g., phase-parallel, divide-and-conquer, pipeline, etc. Each of these paradigms are suited to a specific class of problems, and as we will see in the following sections, there are many trade-offs not only behind selecting the strategy that best suits a problem, but also when fine-tuning it to extract the best performance from a system.

For the Mandelbrot problem, we used a paradigm called **Divide and Conquer**, which is a data parallelism technique in which the data is divided in chunks and distributed across processes for execution. In this setting, each process execute the same piece of code, but in different chunks of data. It is a very useful paradigm, in which there is no dependency between the data we want to process. We find this paradigm well-suited to our problem, as we discussed in Section 1.1, there is no dependency between the pixels to perform the calculations, therefore we can parallelize the pixel-wise computations by assigning a subset of pixels to each process and let them perform the computations independently.

As mentioned previously, we relied on the OpenMP library for the distributed code implementation. OpenMP is an API specification for parallel computing provided for C, C++ and Fortran programming languages. There are a couple of advantages of using OpenMP such as:

- Built-in availability: OpenMP is embedded into the compilers so no overhead configuration is required.
- Ease of use: It provides a set of *directives* to extend the code with parallel functionality.
- Extendable: It allows for extending a serial code to parallel with almost no hard changes.

2.1. Static Scheduling

The first strategy that we adopted was a static scheduling approach. This strategy employs a static division of work between processes, i.e., each process is assigned to a subset of the data where each subset of data has approximately the same size. From this, we divided the pixels in a column-wise fashion, therefore every process operated over all lines from the range of columns of pixels it was assigned to. A high level parallel implementation is provided in Algorithm 2. As we discussed in Section 2 OpenMP does not require huge modifications to code to add parallelism, the only modification was in outer for loop range to compute only the pixel columns for a given thread number.

Algorithm 2 Static scheduling code

```
1:  procedure StaticScheduling(numberOfThreads)
2:      chunkSize ← width/numberOfThreads
3:      threadNumber ← getThreadNumber() ▷ Get threadNumber from OpenMP's API
4:      for i = threadNumber × chunkSize to (threadNumber + 1) × chunkSize do
5:          for j = 0 to height − 1 do
6:              x ← x_min + i × (x_max − x_min)/width
7:              y ← y_min + j × (y_max − y_min)/height
8:              c ← x + iy
9:              z ← 0
10:             n ← 0
11:             while |z| ≤ 2 and n < maxIter do
12:                 z ← z^2 + c
13:                 n ← n + 1
14:             end while
15:             Store n as the color value for pixel (i, j)
16:         end for
17:     end for
18: end procedure
```

This technique implied a problem we did not know in advance. We empirically discovered that some threads working on specific chunks of the image always finished before other threads. Because of that, once the job was done those threads have become idle for a significant amount of time. In other words, the execution time was bounded by the thread that did the most significant amount of computation, leading the other threads to be unused after finishing the work it was assigned to them. We concluded that because of the Mandelbrot algorithm's nature, pixels located in some regions of the image required more iterations to escape, because of that, threads assigned for those cases tended to perform more computations. An illustration of the impacts of this scheduling strategy is depicted in Figure 1, where threads with IDs 0, 4, 5 and 9 execute the algorithm for a negligible amount of time in contrast to the other ones (threads: 1,2,6,7 and 8). Yet, the workload distribution between the threads that computed for a longer time was very irregular, which means, unused resources.

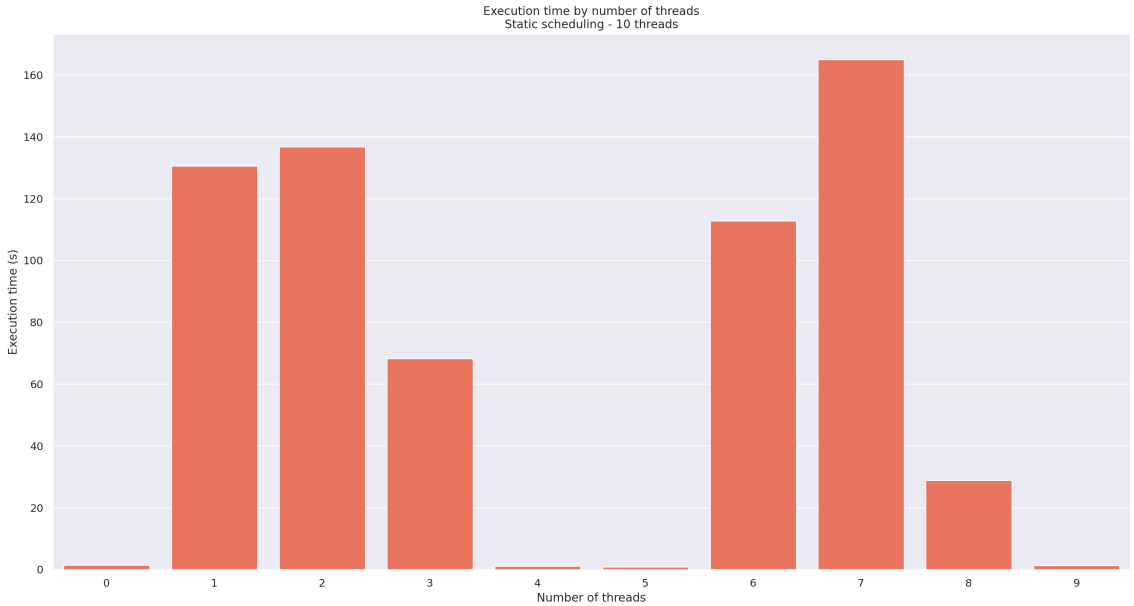


Figure 1. Execution time per thread using static scheduling - Grain size of 768 columns

2.2. Dynamic Scheduling

The problem with static scheduling illustrates what we previously mentioned in Section 2, that in spite of finding the most suitable parallelization paradigm for a task at hand, fine-tuning it to extract the best performance of a system is a tricky task that involves many trade-offs, as we will see along the discussion about the dynamic scheduling approach.

In order to prevent these adversities associated to the static scheduling approach, we adopted the dynamic scheduling strategy. This approach involves reducing data granularity, i.e., the amount of work associated to each process, and distributing more work for the processes, as the task that was associated to them gets done, and there is no work to be done anymore. In other words, the tasks are distributed to the process on the fly, as the threads finishes the work that was assigned to them.

For dynamic scheduling to work, we needed to change our algorithm to make each thread reads a part of the image that no other thread have already read to guarantee each chunk is processed only once. To achieve this, we had to create a control variable (shared by all threads) that holds the value for the chunk that should be processed from the image. Then, when a thread finishes, it reads the current chunk index and update it for the next thread to work with. The problem with this approach is that it may introduce concurrency issues, as we have to prevent threads from trying updating the control variable at the same time. To accomplish this, we constrained the control variable memory address to be considered as a critical region, which OpenMP guarantees that only one thread is updating the control variable value at time. The problem with that approach is that it also prevents other threads from reading from that memory region as well, which may create a barrier if a process finishes its job whereas another one is reading or writing from that controlled region. This issue could be greater if various threads finish at the same time because all threads must wait while one is reading or writing. Yet, we still expect it to guarantee more uniform usages of computational resources in contrast to the static approach depicted in

Section 2.1.

Algorithm 3 shows the updated parallel code with a shared control variable $sharedChunkIndex$. Both lines 2 and 3, from the Algorithm 3, are inside a critical region, so only a single thread can read and update the $sharedChunkIndex$ at a time.

Algorithm 3 Dynamic scheduling code

```
1:  procedure DynamicScheduling(numberOfThreads, chunkSize)
2:      chunkIndex ← sharedChunkIndex                                  ▷ Critical
3:      sharedChunkIndex ← sharedChunkIndex + 1                        ▷ Critical
4:      for i = chunkIndex × chunkSize to (chunkIndex + 1) × chunkSize do
5:          for j = 0 to height − 1 do
6:              x ← x_min + i × (x_max − x_min) / width
7:              y ← y_min + j × (y_max − y_min) / height
8:              c ← x + iy
9:              z ← 0
10:             n ← 0
11:             while |z| ≤ 2 and n < maxIter do
12:                 z ← z^2 + c
13:                 n ← n + 1
14:             end while
15:             Store n as the color value for pixel (i, j)
16:         end for
17:     end for
18: end procedure
```

By applying dynamic scheduling to our problem, we were able to improve efficiency of threads by making each thread process in random regions based on data availability, even if a thread is working on a chunk that requires more iterations to finish, so each thread process almost at the same time as depicted in Figure 2.

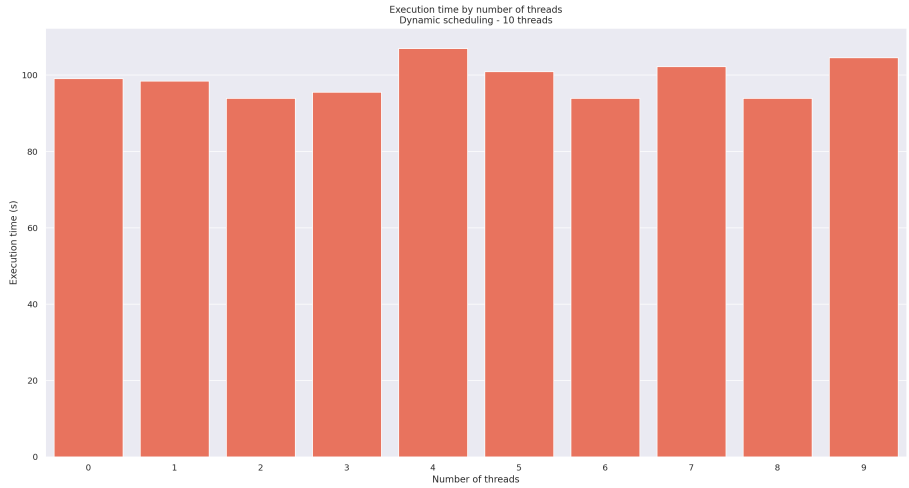


Figure 2. Execution time per thread dynamic scheduling - Grain size 100 columns

3. Experiments

We have performed several experiments logging time of execution for each combination of parameters. We have fixed the image size to a resolution of 8K, meaning output image have 7680×4320, columns and rows respectively, resulting in more than 3 millions pixels.

For serial and parallel executions we evaluated different configurations of maximum amount of iterations, namely: 10, 100, 1000 and 10000. Whereas for the parallel execution, the number of threads assessed ranged between 1 and 12. In addition to that we also evaluated the algorithm for 100000 iterations but only for the serial and 10 and 12 threads configurations for practical reasons. When using dynamic scheduling, it is also possible to vary the grain size assigned to each thread. In our experiments, we tested some grain size configurations. In contrast, static scheduling assigns an equal workload (i.e., grain size) to each thread by default. As discussed in Section 2.1, using a large grain size can negatively affect performance. Therefore, we conducted experiments with grain sizes of 10 and 100. For all configurations, we measured only the execution time of the Mandelbrot set computation, excluding the time required to write the output image to disk.

3.1. Hardware Specifications

Our experiments were executed in a 12th Gen Intel(R) Core(TM) i7-1255U central processing unit (CPU). A summary of the hardware specifications is described as follows:

- 12th Gen Intel(R) Core(TM) i7-1255U [Intel® 2024]
- Number of cores: 10
- Performance cores: 2
- Efficient cores: 8
- Number of threads: 12
- Frequency: 3.5, up to 4.7 (GHz)

In addition to the CPU specifications, the system's hardware configuration also includes the following components:

- Memory (RAM): 16 GB
- Storage: 512 GB SSD

3.2. System Specifications

All experiments were conducted on a system running Ubuntu 22.04.1, a Linux-based operating system. The system configuration is detailed below:

- Operating System: Ubuntu 22.04.1 LTS
- Kernel Version: 6.8.0-57-generic
- System Architecture: x86_64
- GCC Version: 11.4 (GNU Compiler Collection)
- OpenMP Version: 4.5

3.3. Execution

To minimize potential bias introduced by system activity, all experiments were conducted on an idle machine, with no other processes running aside from the operating system. We developed a simple script to automate the execution of our parallel implementations across different configurations of thread counts and iteration values. The script systematically fixed the number of iterations while varying the number of threads, ensuring that all combinations were executed in a controlled and consistent manner. Each execution was logged to a CSV file, recording the number of threads, the maximum number of iterations, and the corresponding execution time in seconds.

We utilized the timing function provided by the OpenMP API, which returns the wall-clock execution time in seconds. This approach offers a straightforward and portable method for measuring the runtime of parallel code segments.

4. Results and Discussions

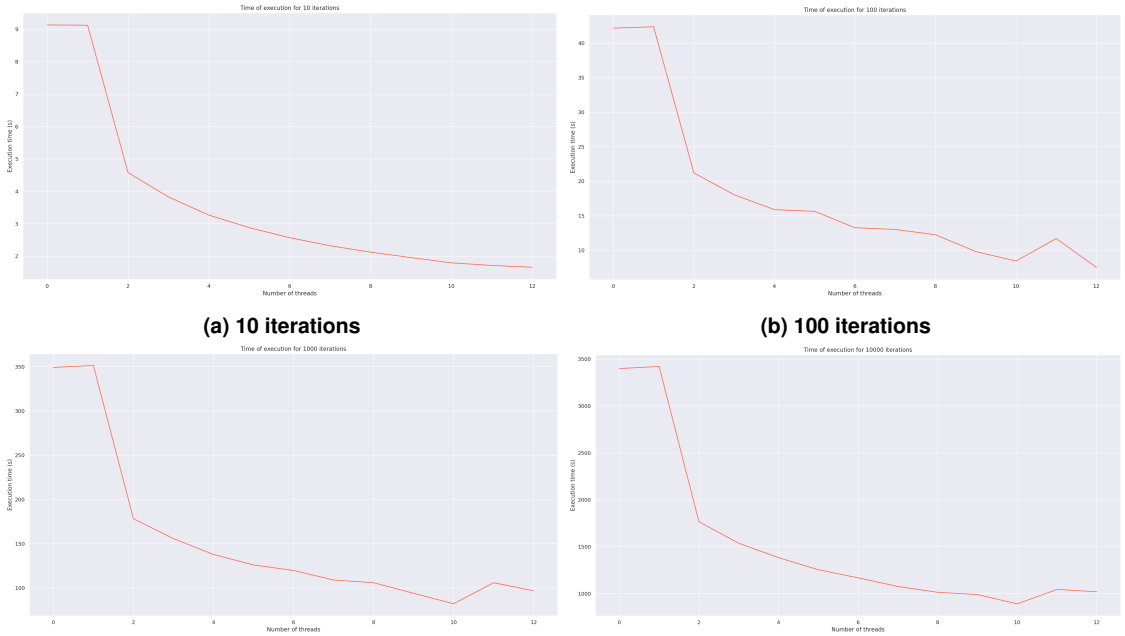
Table 1 demonstrates the precise execution time for each experimental configuration. As mentioned before, we scaled the problem in terms of the number of iterations (N) and number of processors (P).

	Number of iterations				
Number of threads	**10**	**100**	**1000**	**10000**	**100000**
Serial	9.13	42.17	349.12	3398.30	33871.42
1	9.12	42.36	351.22	3420.71	-
2	4.58	21.20	178.23	1766.63	-
3	3.83	18.04	155.74	1537.23	-
4	3.27	15.88	137.77	1383.79	-
5	2.88	15.63	126.01	1255.24	-
6	2.57	13.26	119.73	1168.78	-
7	2.32	13.01	108.99	1076.57	-
8	2.12	12.25	105.94	1014.26	-
9	1.96	9.81	93.94	990.37	-
10	1.79	8.43	**82.14**	**891.0**7	**9454.07**
11	1.71	11.69	105.87	1044.74	-
12	**1.66**	**7.51**	96.80	1019.81	10166.71

Table 1. Execution time, in seconds, by number of threads and maximum number of iterations

Figure 3 demonstrates the execution time (in seconds) by number of threads for each configuration of iterations. As we can see, as the number of iterations increases, thus the complexity of the problem and demand for computational resources, adding more threads becomes inefficient. This is justified when we compare Figure 3 **a)**, in which adding more threads always reduces the execution time, whereas in Figure 3 **b)**, running with 11 and 12 threads takes longer that running with 10. From our knowledge, this was expected since as we increase the allocation of computational resources for executing this program, it begins to compete with the operating system which leads to a decrease in efficiency. For fewer iterations e.g., 10 this doesn't occur, which makes sense, as we see

that the amount of time that this program allocated all resources was inferior to 1 second. But as the problem size becomes less negligible (N increases), we start to see that the dispute for resources gets more significant.



(a) 10 iterations **(b) 100 iterations**

(c) 1000 iterations **(d) 10000 iterations**

Figure 3. Execution time per number of threads. Zero threads means serial execution

We have tested few different grain sizes trying to find a good alternative instead of equally splitting our problem. When using a too small grain size such as 10, the algorithms yield a high amount of communication between threads to prevent reading and writing into the same memory address which causes almost every execution to take longer time than a large grain size, in contrast, using a grain size too large make our program to get closer to static scheduling where some threads execute more than others. Through few attempts, we identified that an interesting grain size for our problem is 100 columns at a time for each thread.

Based on the execution times presented in Table 1, we compute the speedup achieved by the parallel versions relative to the serial implementation. The speedup is defined in Equation 1, where T_s denotes the execution time of the serial version, T_p is the execution time of the parallel version, and p represents the number of processors used.

$$S(p) = \frac{T_s}{T_p} \tag{1}$$

In Figure 4, we can see the speedup of each problem in terms of the number of iterations executed and number of threads. We observed that speedup was inversely proportional to the number of iterations. Among the tested configurations, 10 iterations generally yielded the highest speedup, whereas 100,000 iterations resulted in the lowest speedup.

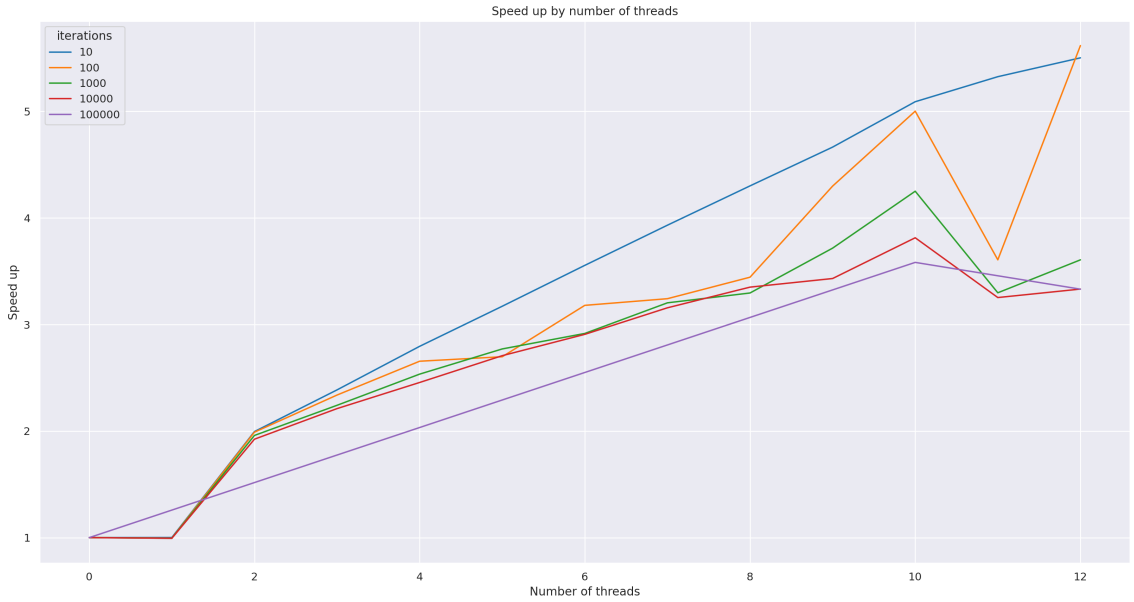


Figure 4. Speed up per thread count

In addition to evaluating the speedup achieved by our parallel executions, we also assessed the efficiency of each implementation under different configurations. Efficiency reflects how effectively computational resources are utilized to solve a given problem. It is defined by Equation 2, where $S(p)$ represents the speedup obtained with p processors, and p is the number of processors used.

$$E(p) = \frac{S(p)}{p} \tag{2}$$

Figure 5 shows that efficiency decreases exponentially as the number of threads increases. In an ideal scenario, efficiency would remain equal to 1 regardless of the number of threads, indicating perfect scalability. In our experiments, this ideal was closely approached only when using two threads, where efficiency was nearly 1. For all configurations with four or more threads, efficiency dropped below 70%, indicating suboptimal utilization of computational resources.

5. Conclusion

This report presented the development, execution, and analysis of both serial and parallel implementations of the Mandelbrot set using the escape-time algorithm. A series of experiments were conducted to evaluate the impact of two key factors on execution time: the maximum number of iterations allowed per pixel during the escape-time calculation, and the number of threads used during parallel execution.

Our findings indicate that the problem is well-suited for parallelization at the pixel level where each thread receives a piece of the problem to process independently. Although this problem can take advantage of using more computational power to improve execution time, we have identified that the Mandelbrot problem is not scalable at every situation. The execution time does not remain constant when increasing the number of iterations, causing our solutions to become inefficient for a large number of configurations.

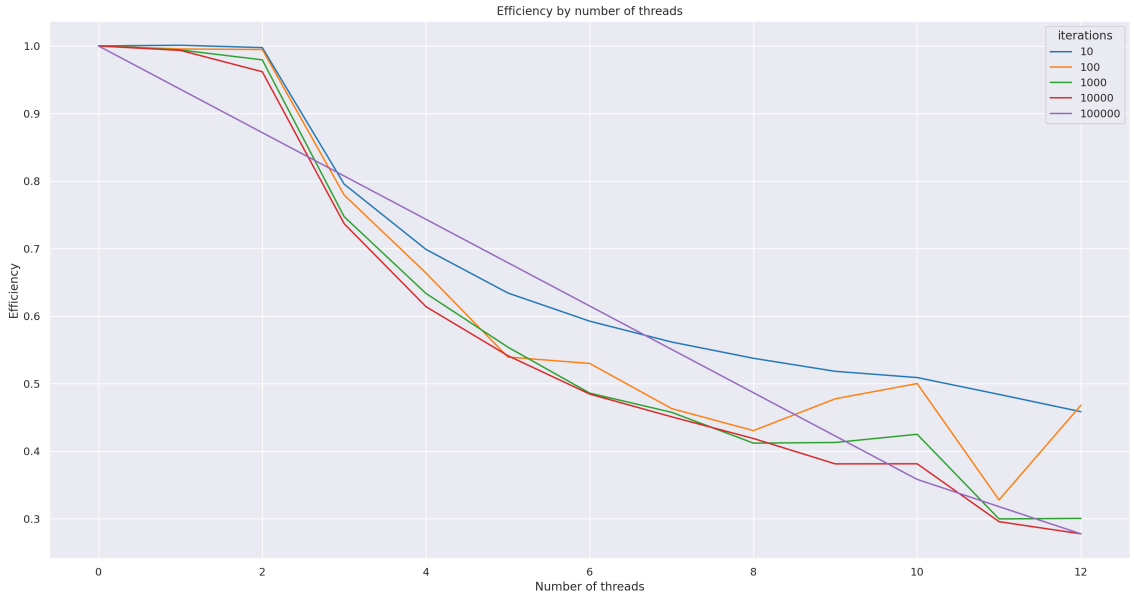


Figure 5. Efficiency per thread count

An interesting discovery we have made is that the execution time when execution the program using 10 threads is faster than executing using 11 and 12 threads. There are a couple of reasons this can happen, as soon as the number of used threads get closer to 12, there is a higher chance for our program to concur with the operational system for resources, the operational system can assign a process to a thread already being used by another process, cache misses because threads can be working on different areas of an image an so on.

Even with these setbacks, we were able to see, validate and analyze the advantages of identifying pieces of a problem that can be executed in parallel to reduce the execution time and use resources available by hardware.

References

Intel® (2024). What is performance hybrid architecture?